

Multi-dimensional grids and blocks

GPU Programming – M2 DiPaC, Université Paris-Saclay – Session 2

Oguz Kaya

Assistant Professor

Université Paris-Saclay and LISN, Gif-sur-Yvette, France

Objectives

- Use 2D and 3D indices for blocks in a grid and for threads in a block (**dim3**)
- Map a 2D array onto blocks and threads in several ways – first with 1D indices only, then with 2D indices – and know how a 2D block is cut into warps

Outline

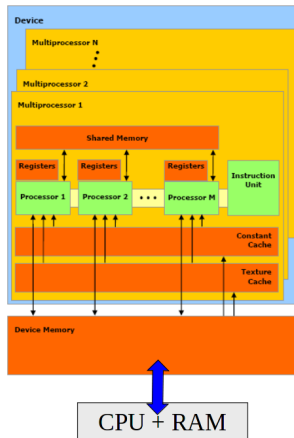
- 1 Recap of session 1
- 2 A 2D array on a 1D grid
- 3 2D grids and 2D blocks
- 4 3D grids and 3D blocks

- 1 Recap of session 1
- 2 A 2D array on a 1D grid
- 3 2D grids and 2D blocks
- 4 3D grids and 3D blocks

Recap: GPU architecture

A GPU is a collection of N independent SIMD processors sharing a global memory

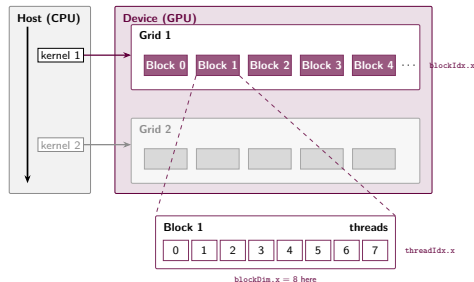
- N streaming “multiprocessors” (SM)
- Each SM is a SIMD processor having
 - k synchronized processors ($k = 32$), in other words, **CUDA cores**
 - 1 shared instruction decoder
 - 3 types of memories shared among all k processors
 - 64K registers per SM, distributed among the threads (up to 255 per **thread**)
 - To fully utilize each SM, you need to launch **at least 32 threads (per block)**
 - To fully utilize the **whole GPU**, you need **at least one block per SM** (in practice several, so the scheduler has work in reserve)



Recap: threads, blocks and grids

The CPU launches a kernel; the GPU runs it with a set of threads.

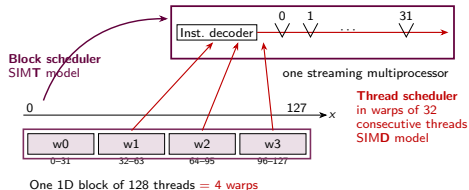
- All threads are identical: they execute the same code
- Threads are grouped into **blocks** (1 to 1024 threads; in practice a multiple of 32)
- Each block runs on one SM; the blocks of a launch form a **grid**, spread over all SMs
- Launch enough blocks and threads to cover the whole iteration domain of the problem
- Threads of the **same block** can share resources and synchronise. **Threads of different blocks cannot.**



Recap: warps, the unit of execution

Make blocks out of a whole number of **warps**

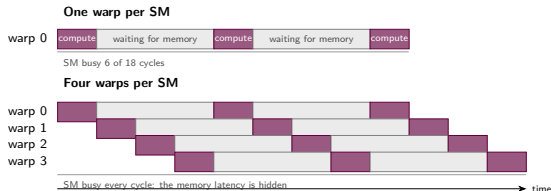
- One instruction decoder drives 32 CUDA cores, i.e. 32 threads at once
- Each group of 32 consecutive threads of a block is a **warp**: the unit the hardware actually schedules
- The threads of a warp are **synchronous**: they execute the same instruction at the same time, each on its own data
- The SM's scheduler picks a warp of a resident block and issues its next instruction to the 32 cores
- A block of 100 threads is 4 warps, the last one 7/8 empty (4 of 32 threads used) – hence “a multiple of 32”



Recap: hiding memory latency

A global-memory access takes hundreds of cycles. The SM does not wait: it switches warps.

- All resident warps keep their registers in the SM, so switching between them costs **nothing**
- While one warp waits for memory, the scheduler issues instructions from another
- With enough warps in flight, the memory latency is **hidden** and the SM stays busy
- So: many **light threads**, many **blocks** – little work per thread, and let the hardware interleave them



Recap: 1D kernels and the global index

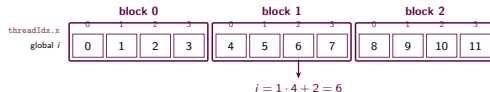
Every thread runs the same kernel. What makes it work on *its* element? Its **global index**:

$$i = \text{blockIdx.x} \cdot \text{blockDim.x} + \text{threadIdx.x}$$

- `threadIdx.x`: the thread's rank in its block,
 $0 \dots \text{blockDim.x} - 1$
- `blockIdx.x`: the block's rank in the grid,
 $0 \dots \text{gridDim.x} - 1$
- Launch enough threads to cover N elements:
 $\lceil N/BS \rceil$ blocks of BS threads, i.e. round *up*

```
int BS = 256;  
int nb = (N + BS - 1) / BS;  
kernel<<<nb, BS>>>(N, ...);
```

- The last block may overshoot N : guard with
if ($i < N$) in the kernel



Recap: a complete 1D kernel

Multiply each element of an array **A[N]** by a scalar **c**.

- **__device__** defines a global array in GPU memory.
- **__global__** defines the function on the GPU.
 - This allows using **blockIdx.x**, **blockDim.x**, and **threadIdx.x**
- Each thread and block always executes the same code.
- We use **blockSize** threads per block.
- Each thread performs 1 operation, so $\lceil N / \text{blockSize} \rceil$ blocks must be launched to cover the entire array/computation domain.
- Execution is differentiated by **blockIdx.x** and **threadIdx.x**.
- Be careful of array overflow (if N is not divisible by blockSize): guard with `if (i < n)`

```
#include <stdio>
#include <cuda_runtime.h>

#define N 1024
float A[N];
float c = 2.0f;

__device__ float dA[N];

__global__ void multiplyArray(int n, float c)
{
    int i = threadIdx.x + blockIdx.x * blockDim.x;
    if (i < n)
        dA[i] *= c;
}

int main(void)
{
    // Initialization
    for (int i = 0; i < N; i++) { A[i] = i; }
    // Copy the array to the GPU
    cudaMemcpyToSymbol(dA, A, N * sizeof(float), 0,
        cudaMemcpyHostToDevice);
    int blockSize = 128;
    int numBlocks = (N + blockSize - 1) / blockSize;
    multiplyArray<<<numBlocks, blockSize>>>(N, c);
    // Copy the multiplied array back to the CPU
    cudaMemcpyFromSymbol(A, dA, N * sizeof(float), 0,
        cudaMemcpyDeviceToHost);
    printf("%f\n", A[2]);
    return 0;
}
```

- 1 Recap of session 1
- 2 A 2D array on a 1D grid
- 3 2D grids and 2D blocks
- 4 3D grids and 3D blocks

A 2D array with a 1D grid: the question

Last session: multiply each element of a 1D array $A[N]$ by c , with a 1D grid of 1D blocks.

Now the array is 2D, $A[N][N]$, and we still want to multiply every element by c – using only `blockIdx.x` and `threadIdx.x`, the 1D indices you know.

- 1 What is the task of **one thread**?
- 2 How many threads do we launch **in total**?
- 3 How do we **regroup** the threads into blocks of `blockSize`? What does *one block* compute – what are the possibilities? And, in each case, how does a thread compute i and j ?

Hint for step 3: think of how A is laid out in memory.

Row-major storage: where $A[i][j]$ lives in memory

Before answering: C stores a 2D array **row after row**, and every index formula of today relies on it.

$$\text{address of } A[i][j] = \text{base} + (i \cdot N + j) \cdot \text{sizeof}(\text{float})$$

- Consecutive j (same row): 4 bytes apart – **contiguous**
- Consecutive i (same column): $4N$ bytes apart – **strided**
- A 2D array is really a 1D array of N^2 elements: this is what makes a 1D thread index usable
- Keep this picture in mind: next session it decides which kernels are fast

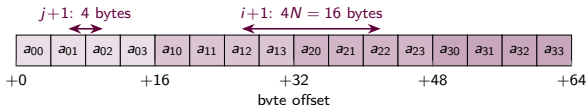
A with $N = 4$, as we think of it

$i = 0$	a_{00}	a_{01}	a_{02}	a_{03}
$i = 1$	a_{10}	a_{11}	a_{12}	a_{13}
$i = 2$	a_{20}	a_{21}	a_{22}	a_{23}
$i = 3$	a_{30}	a_{31}	a_{32}	a_{33}

row i starts at byte $i \cdot 4N$
element j of it is $4j$ bytes further

so `float A[N][N]` and `float A[N*N]`
(indexed `A[i*N+j]`) are the same bytes

$j = 0 \quad 1 \quad 2 \quad 3$
the same bytes in memory: one row after another

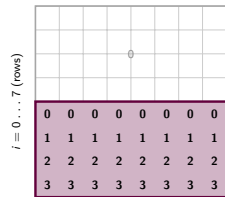


A first answer: one thread per row

- **Step 1:** the task of one thread is to multiply an **entire row** $A[i][.]$ – a loop over j inside the kernel
- **Step 2:** N threads in total, one per row
- **Step 3:** this is exactly the 1D problem of session 1 – $\lceil N/\text{blockSize} \rceil$ blocks, the usual global index i , guard with $\text{if } (i < n)$
- Correct, but with $N = 128$ rows we would launch only 128 threads – we cannot fill a GPU this way (session 1: many light threads)

```
--global__ void multiplyArray2D(int n, float c)
{
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) {
        for (int j = 0; j < n; j++) { dA[i][j] *= c; }
    }
    ...
    int blockSize = 256;
    int numBlocks = (N + blockSize - 1) / blockSize;
    multiplyArray2D<<<numBlocks, blockSize>>>(N, c);
}
```

$j = 0 \dots 7$ (columns) [blockSize = 4]



grey: blockIdx.x ; highlighted: block 1
each thread multiplies a whole row;
cells: threadIdx.x ; $i = 1 \cdot 4 + \text{threadIdx.x}$

One element per block/thread, N^2 blocks: the other order

- Nothing forces the row order: let each group of N consecutive blocks cover one **column** of A instead
- Division and modulus swap roles:
 $j = \text{blockIdx.x} / N$,
 $i = \text{blockIdx.x} \bmod N$
- Both versions are correct and launch the same N^2 blocks; consecutive blocks now walk down a column

```
__global__ void multiplyArray2D(int n, float c)
{
    int i = blockIdx.x % n;
    int j = blockIdx.x / n;
    dA[i][j] *= c;
}

...
multiplyArray2D<<<N * N, 1>>>>(N, c);
```

$j = 0 \dots 7$ (columns) [blockSize = 4]

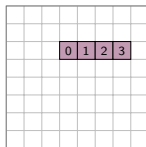
	0	8	16	24	32	40	48	56
	1	9	17	25	33	41	49	57
	2	10	18	26	34	42	50	58
	3	11	19	27	35	43	51	59
	4	12	20	28	36	44	52	60
	5	13	21	29	37	45	53	61
	6	14	22	30	38	46	54	62
	7	15	23	31	39	47	55	63

$i = 0 \dots 7$ (rows)

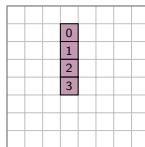
each cell: one block of one thread, labelled blockIdx.x
highlighted: block 11 computes $A[3][1]$
 $i = 11 \bmod 8 = 3$, $j = 11 / 8 = 1$

One element per thread, multiple threads per block

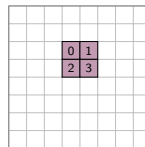
With `blockSize` threads per block, a block covers `blockSize` elements of `A`. Which ones? (example for `blockSize = 4`)



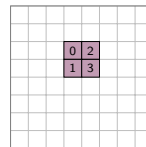
(a) 1D block:
a piece of a **row**



(b) 1D block:
a piece of a **column**



(c) block = 2×2 **tile**, threads row after row



(d) block = 2×2 **tile**, threads column after column

- (a), (b): a 1D block is a piece of a row, or a piece of a column
- (c), (d): a block is a square **tile**; its threads can be laid out in the tile *row after row* or *column after column*
- All four are correct; with 1D indices each needs some division/modulus. **Find the formulas for i and j** before the next four slides

(a) 1D blocks along a row, 1D grid

- A row needs $\lceil N/\text{blockSize} \rceil$ blocks (`blocksPerRow`); the grid is $N \cdot \text{blocksPerRow}$ blocks, so the kernel recovers `blocksPerRow` as `gridDim.x / n`

- Block coordinates:
 $b_i = \text{blockIdx.x} / \text{blocksPerRow}$
 (which row), $b_j =$
 $\text{blockIdx.x} \bmod \text{blocksPerRow}$
 (which piece of it)

- Element: $i = b_j$,
 $j = b_j \cdot \text{blockDim.x} + \text{threadIdx.x}$
 – the 1D global-index formula,
 inside the row

- The last piece of a row may overshoot N : guard with `if (j < n)`

```
__global__ void multiplyArray2D(int n, float c)
{
    int blocksPerRow = gridDim.x / n;
    int bi = blockIdx.x / blocksPerRow;
    int bj = blockIdx.x % blocksPerRow;
    int i = bi;
    int j = bj * blockDim.x + threadIdx.x;
    if (j < n) { dA[i][j] *= c; }
}

...
int BS = 256;
int blocksPerRow = (N + BS - 1) / BS;
int nb = N * blocksPerRow;
multiplyArray2D<<<nb, BS>>>(N, c);
```

$j = 0 \dots 7$ (columns) [blockSize = 4]

$i = 0 \dots 7$ (rows)	0			1		
	2		0	1	2	3
	4				5	
	6				7	
	8				9	
	10				11	
	12				13	
	14				15	

```
grey: blockIdx.x; highlighted: block 3
 $b_i = 3/2 = 1, b_j = 3 \bmod 2 = 1$ 
cells: threadIdx.x;  $i = b_i, j = b_i \cdot 4 + \text{threadIdx.x}$ 
```

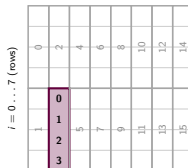
(b) 1D blocks along a column, 1D grid

- Same idea with the roles of i and j swapped: $\lceil N/\text{blockSize} \rceil$ blocks per column
- Block coordinates:
 $b_j = \text{blockIdx.x} / \text{blocksPerCol}$
 (which column), $b_i = \text{blockIdx.x} \bmod \text{blocksPerCol}$
 (which piece of it)
- Element: $i = b_i \cdot \text{blockDim.x} + \text{threadIdx.x}$,
 $j = b_j$; guard on i
- Which one is better – (a) or (b)?
 Keep the question in mind

```
__global__ void multiplyArray2D(int n, float c)
{
    int blocksPerCol = blockDim.x / n;
    int bj = blockIdx.x / blocksPerCol;
    int bi = blockIdx.x % blocksPerCol;
    int i = bi * blockDim.x + threadIdx.x;
    int j = bj;
    if (i < n) { dA[i][j] *= c; }
}

...
int BS = 256;
int blocksPerCol = (N + BS - 1) / BS;
int nb = N * blocksPerCol;
multiplyArray2D<<<nb, BS>>>>(N, c);
```

$j = 0 \dots 7$ (columns) [blockSize = 4]



grey: blockIdx.x; highlighted: block 3
 $b_j = 3/2 = 1$, $b_i = 3 \bmod 2 = 1$
 cells: threadIdx.x ; $i = b_i \cdot 4 + \text{threadIdx.x}$, $j = b_j$

(c) A square tile per block, threads row after row

- **blockSize** must be a **square number**: the tile has side $\sqrt{\text{blockSize}}$ ($1024 \rightarrow 32 \times 32$); the kernel recovers it from `blockDim.x`
- $\lceil N/\text{side} \rceil^2$ blocks; two levels of division/modulus: `blockIdx.x` $\rightarrow$ block coordinates (b_i, b_j) , `threadIdx.x` $\rightarrow$ thread coordinates (t_i, t_j) in the tile
- Row after row:

$$t_i = \text{threadIdx.x} / \text{side},$$

$$t_j = \text{threadIdx.x} \bmod \text{side};$$
 then $i = b_i \cdot \text{side} + t_i$,
 $j = b_j \cdot \text{side} + t_j$; guard on both
- **Where do the warps end up?**
 - With `side = 32`, a warp is 32 consecutive `threadIdx.x = one row` of the tile

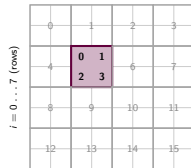
```

__global__ void multiplyArray2D(int n, float c)
{
    int side = sqrtf(blockDim.x);
    int tilesPerRow = (n + side - 1) / side;
    int bi = blockIdx.x / tilesPerRow;
    int bj = blockIdx.x % tilesPerRow;
    int ti = threadIdx.x / side;
    int tj = threadIdx.x % side;
    int i = bi * side + ti;
    int j = bj * side + tj;
    if (i < n && j < n) { dA[i][j] *= c; }
}

...

int blockSize = 1024;
int side = sqrtf(blockSize);
int tilesPerRow = (N + side - 1) / side;
int nb = tilesPerRow * tilesPerRow;
multiplyArray2D<<<nb, blockSize>>>(N, c);

```



grey: blockIdx.x; highlighted: block 5
 $b_i = 5/4 = 1$, $b_j = 5 \bmod 4 = 1$; cells: threadIdx.x
 $t_i = \text{threadIdx.x}/2$, $t_j = \text{threadIdx.x} \bmod 2$

(d) A square tile per block, threads column after column

- Same tile;
 - $t_i = \text{threadIdx.x} \bmod \text{side}$,
 - $t_j = \text{threadIdx.x} / \text{side}$:
 consecutive threads go down a column of the tile
- Where do the warps end up?
 - A warp is now one **column** of the tile
- Which one is better – (c) or (d)?
- All four answers work – at the price of division/modulus arithmetic in every kernel: for the block in (a)–(d), and for the thread too in (c), (d)
- CUDA can index blocks and threads in 2D directly, and that arithmetic disappears: **next section**

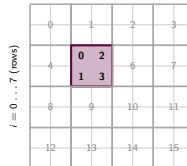
```

__global__ void multiplyArray2D(int n, float c)
{
    int side = sqrtf(blockDim.x);
    int tilesPerRow = (n + side - 1) / side;
    int bi = blockIdx.x / tilesPerRow;
    int bj = blockIdx.x % tilesPerRow;
    int ti = threadIdx.x % side;
    int tj = threadIdx.x / side;
    int i = bi * side + ti;
    int j = bj * side + tj;
    if (i < n && j < n) { dA[i][j] *= c; }
}

...
int blockSize = 1024;
int side = sqrtf(blockSize);
int tilesPerRow = (N + side - 1) / side;
int nb = tilesPerRow * tilesPerRow;
multiplyArray2D<<<nb, blockSize>>>(N, c);

```

$j = 0 \dots 7$ (columns) [blockSize = 4]



grey: blockIdx.x; highlighted: block 5
 $b_i = 5/4 = 1$, $b_j = 5 \bmod 4 = 1$; cells: threadIdx.x
 $t_i = \text{threadIdx.x} \bmod 2$, $t_j = \text{threadIdx.x} / 2$

- 1 Recap of session 1
- 2 A 2D array on a 1D grid
- 3 2D grids and 2D blocks**
- 4 3D grids and 3D blocks

A 2D array, one block per element, 2D grid

- **dim3**: a type with three members **x, y, z**, describing a 3D layout of blocks in a grid (later: of threads in a block)

```
__global__ void multiplyArray2D(int n, float c)
{
    int i = blockIdx.x;
    int j = blockIdx.y;
    dA[i][j] *= c;
}
```

- `dimGrid.x = dimGrid.y = N`, `dimGrid.z = 1`: an $N \times N$ grid of `dimGrid.x * dimGrid.y * dimGrid.z` blocks. Unset members are 1; `dim3 dimGrid(N, N)` is the short form

```
...
dim3 dimGrid;
dimGrid.x = N;
dimGrid.y = N;
dimGrid.z = 1;
multiplyArray2D<<<dimGrid, 1>>>>(N, c);
```

- The block reads i, j directly from **blockIdx.x**, **blockIdx.y**: no division or modulus
- With `blockSize` threads per block (next slides), rounding up applies **per dimension**: $\lceil N/\text{blockSize} \rceil$ blocks in x , in y , ..., and the kernel guards each index

$j = 0 \dots 7$ (columns) [blockSize = 4]

0,0	0,1	0,2	0,3	0,4	0,5	0,6	0,7
1,0	1,1	1,2	1,3	1,4	1,5	1,6	1,7
2,0	2,1	2,2	2,3	2,4	2,5	2,6	2,7
3,0	3,1	3,2	3,3	3,4	3,5	3,6	3,7
4,0	4,1	4,2	4,3	4,4	4,5	4,6	4,7
5,0	5,1	5,2	5,3	5,4	5,5	5,6	5,7
6,0	6,1	6,2	6,3	6,4	6,5	6,6	6,7
7,0	7,1	7,2	7,3	7,4	7,5	7,6	7,7

$i = 0 \dots 7$ (rows)

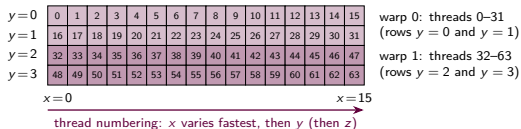
each cell: one block of one thread,
labelled (blockIdx.x, blockIdx.y)
highlighted: block (1, 5) computes $A[1][5]$

2D blocks: how the hardware numbers the threads

Threads can be indexed in 2D or 3D too (`dim3 dimBlock`); the hardware still sees them as a **sequence**.

- Within a block, threads are ordered with **`threadIdx.x` varying fastest**, then `threadIdx.y`, then `threadIdx.z`
- That sequence is cut into **warps of 32**: for `blockDim.x ≥ 32`, a warp is 32 threads with consecutive `threadIdx.x` and the same `y, z`
- A fact about the hardware, not a choice of the programmer – whatever *i* and *j* you compute from the indices
- **Remember it**: it is what decides, next session, which of today's kernels are fast

block 16×4 (`blockDim.x = 16`, `blockDim.y = 4`): 64 threads, two warps



block 32×2 : two warps, one per row



(c) 2D grid of 2D blocks: threads row after row

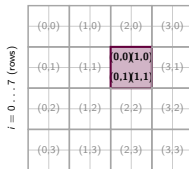
- Case (c) with **2D blocks**: dim3 dimBlock with $\text{dimBlock.x} = \text{dimBlock.y} = 32$ (a 32×32 tile); the thread coordinates are read directly, $t_i = \text{threadIdx.y}$, $t_j = \text{threadIdx.x}$ – no division/modulus at all
- Block coordinates likewise:
 $b_i = \text{blockIdx.y}$,
 $b_j = \text{blockIdx.x}$; then
 $i = b_i \cdot 32 + t_i$, $j = b_j \cdot 32 + t_j$
- Grid: $\lceil N/32 \rceil \times \lceil N/32 \rceil$ blocks, computed from dimBlock.x , dimBlock.y ; guard on both i and j
- Warps (previous slide): 32 consecutive $\text{threadIdx.x} = \text{one row}$ of the tile, so a warp works along a row of A

```
__global__ void multiplyArray2D(int n, float c)
{
    int i = blockIdx.y * blockDim.y + threadIdx.y;
    int j = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n && j < n) { dA[i][j] *= c; }
}

...

dim3 dimBlock;
dimBlock.x = 32;
dimBlock.y = 32;
dimBlock.z = 1;
dim3 dimGrid;
dimGrid.x = (N + dimBlock.x - 1) / dimBlock.x;
dimGrid.y = (N + dimBlock.y - 1) / dimBlock.y;
dimGrid.z = 1;
multiplyArray2D<<<dimGrid, dimBlock>>>(N, c);
```

$j = 0 \dots 7$ (columns) [blockSize = 4]



```
grey: (blockIdx.x, blockIdx.y); highlighted: (2, 1)
cells: (threadIdx.x, threadIdx.y)
i = 1 * 2 + threadIdx.y, j = 2 * 2 + threadIdx.x
```

(d) 2D grid of 2D blocks: threads column after column

- Case (d) with 2D blocks: same block and grid, the roles of x and y swapped – $t_i = \text{threadIdx.x}$,
 $t_j = \text{threadIdx.y}$,
 $b_i = \text{blockIdx.x}$,
 $b_j = \text{blockIdx.y}$
- A warp is now one **column** of the tile: consecutive threads of a warp work down a column of A
- **Which one is better – (c) or (d)?**

```
__global__ void multiplyArray2D(int n, float c)
{
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    int j = blockIdx.y * blockDim.y + threadIdx.y;
    if (i < n && j < n) { dA[i][j] *= c; }
}

...

dim3 dimBlock;
dimBlock.x = 32;
dimBlock.y = 32;
dimBlock.z = 1;
dim3 dimGrid;
dimGrid.x = (N + dimBlock.x - 1) / dimBlock.x;
dimGrid.y = (N + dimBlock.y - 1) / dimBlock.y;
dimGrid.z = 1;
multiplyArray2D<<<dimGrid, dimBlock>>>(N, c);
```

$j = 0 \dots 7$ (columns) [blockSize = 4]

$(0,0)$	$(0,1)$	$(0,2)$	$(0,3)$
$(1,0)$	$(1,1)$	$0.0 0.1$ $1.0 1.1$	$(1,3)$
$(2,0)$	$(2,1)$	$(2,2)$	$(2,3)$
$(3,0)$	$(3,1)$	$(3,2)$	$(3,3)$

```
grey: (blockIdx.x, blockIdx.y); highlighted: (1, 2)
cells: (threadIdx.x, threadIdx.y)
i = 1 * 2 + threadIdx.x, j = 2 * 2 + threadIdx.y
```

The four cases with 2D indexing – which one is fastest?

All four kernels are correct and do the same work. They differ only in **what consecutive threads touch**:

case	block	consecutive threadIdx.x work...
(a)	1D, 1024 threads, along a row	along a row of A
(b)	1D, 1024 threads, along a column	down a column of A
(c)	2D, 32×32 , $t_j = \text{threadIdx.x}$	along a row of A
(d)	2D, 32×32 , $t_i = \text{threadIdx.x}$	down a column of A

- Time them yourself: two of them are several times faster than the other two
- The answer depends on how the 32 threads of a **warp** access memory *together* – **next session**

- 1 Recap of session 1
- 2 A 2D array on a 1D grid
- 3 2D grids and 2D blocks
- 4 3D grids and 3D blocks**

Dimension limits for grids and blocks

For a grid (`dimGrid`):

- $x \leq 2^{31} - 1$
- $y \leq 65535$
- $z \leq 65535$

For a block (`dimBlock`):

- $x \leq 1024$
- $y \leq 1024$
- $z \leq 64$
- $x \cdot y \cdot z \leq 1024$

- x is the “big” dimension of the grid because it is the one a 1D launch uses: 2^{31} blocks of 1024 threads cover any array that fits in memory
- y and z of the grid are for convenience of indexing, not for capacity
- The 1024-thread cap is per block, whatever its shape: 1024×1 , 32×32 , $8 \times 8 \times 8$, ...
- Exceeding a limit is a *launch* error: caught by `cudaGetLastError()` – `CUDA_CHECK_KERNEL()`

And in 3D: a 3D grid of 3D blocks

Multiply each element of $A[N][N][N]$ by c – z follows exactly the same rules as x and y .

- `dim3 dimBlock(8, 8, 8)`: 512 threads per block, a cube of A
- Grid of $\lceil N/8 \rceil^3$ blocks; one global index per dimension, one guard per dimension
- Storage is still “last index fastest”:
 $A[i][j][k]$ is at $((i \cdot N + j) \cdot N + k) \cdot 4$, so `threadIdx.x` is mapped to the **last** index k
- Limits: `dimBlock.z` ≤ 64 , and
 $x \cdot y \cdot z \leq 1024$ threads per block (previous slide)

```
__global__ void multiplyArray3D(int n, float c)
{
    int i = blockIdx.z * blockDim.z + threadIdx.z;
    int j = blockIdx.y * blockDim.y + threadIdx.y;
    int k = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n && j < n && k < n) {
        dA[i][j][k] *= c;
    }
}

...
dim3 dimBlock(8, 8, 8);
int nb = (N + 7) / 8;
dim3 dimGrid(nb, nb, nb);
multiplyArray3D<<<dimGrid, dimBlock>>>(N, c);
```

Contact

Oguz Kaya

Université Paris-Saclay and LISN, Gif-sur-Yvette, France

oguz.kaya@universite-paris-saclay.fr

www.oguzkaya.com

Course material: github.com/kayaogz/2627-m2dipac-gpuprogramming